Solution to analysis in Home Assignment 4

Osama Al Sheikh Ali + Chalmers ID (osamas)

Analysis

In this report I will present my independent analysis of the questions related to home assignment 4. I have discussed the solution with Mubina Crnic, but I swear that the analysis written here are my own.

1 Smoothing

a)

By choosing my favourite filter as in the previous assignment which is EKF. We can observe after looking at the results from the smoothing that it is not ideal since there is a large covariance in the start of the step and it is also bad at the curve. But when incorporating smoothing the estimation becomes way smoother since it uses future data to approximate our current position. This will lead to getting results which fit the real states better because we minimize the side effects of the measurement noise.

$$\text{Forward:} \quad \begin{cases} \mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}_k^T\mathbf{S}_k^{-1} \\ \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k\left(\mathbf{y}_k - \mathbf{H}_k\hat{\mathbf{x}}_{k|k-1}\right) \\ \mathbf{P}_{k|k} = \mathbf{P}_{k|k-1} - \mathbf{K}_k\mathbf{S}_k\mathbf{K}_k^T \end{cases}$$

$$\text{Backward:} \quad \begin{cases} \mathbf{G}_k = \mathbf{P}_{k|k}\mathbf{A}_k^T\mathbf{P}_{k+1|k}^{-1} \\ \hat{\mathbf{x}}_{k|K} = \hat{\mathbf{x}}_{k|k} + \mathbf{G}_k\left(\hat{\mathbf{x}}_{k+1|K} - \hat{\mathbf{x}}_{k+1|k}\right) \\ \mathbf{P}_{k|K} = \mathbf{P}_{k|k} - \mathbf{G}_k\left[\mathbf{P}_{k+1|k} - \mathbf{P}_{k+1|K}\right]\mathbf{G}_k^T \end{cases}$$

Figure 1: Smoothing equations

The results can be seen from the plots below. Looking at the zoomed plot we can observe that the filter estimation differs more than what than smoothed estimation differs from the real positions. It can be observed that the covariance of the EKF is also larger than what the smoothed covariance is even though both do in fact coincide.

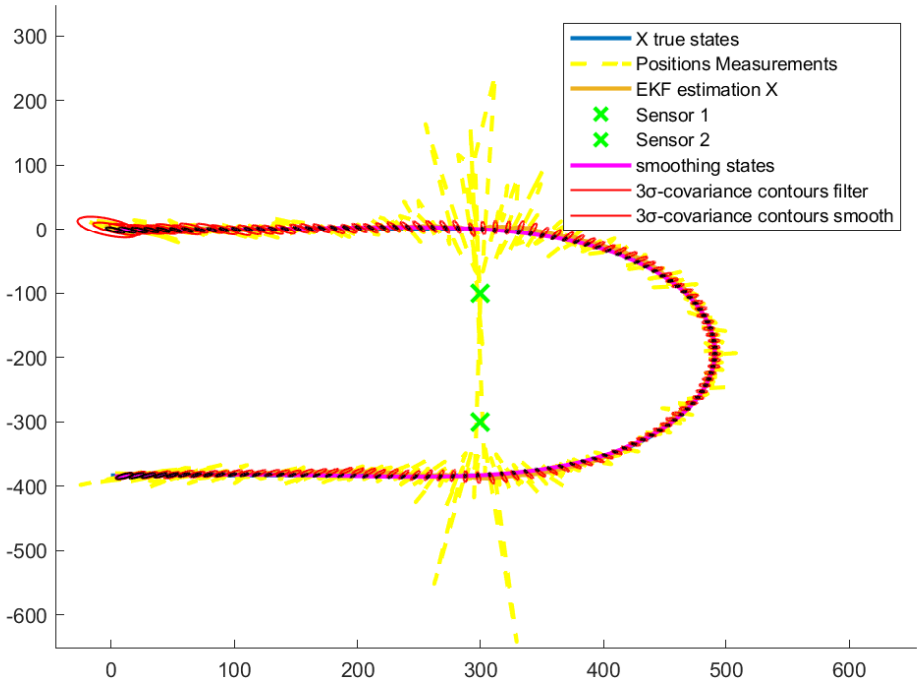


Figure 2: EKF - FILTERED AND SMOOTHED POSITION

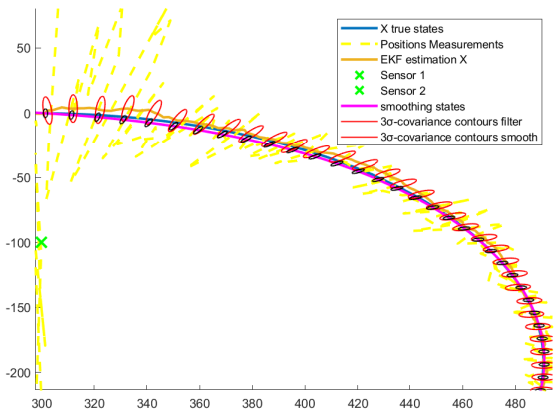


Figure 3: EKF - FILTERED AND SMOOTHED POSITION - ZOOMED

A valuable insight which confirms our conclusions can be seen from the error covariance plots as seen below. Here we can see that EKF has higher error covariance than the smoothed.

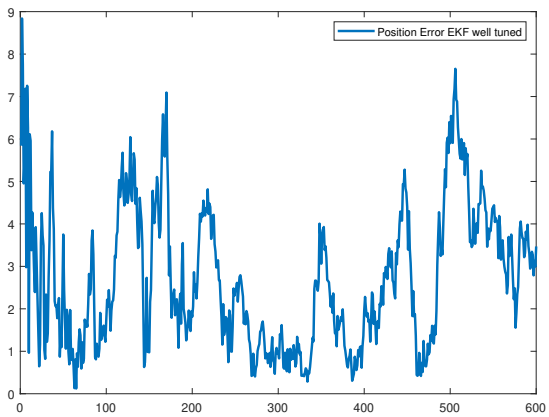


Figure 4: EKF error covariance

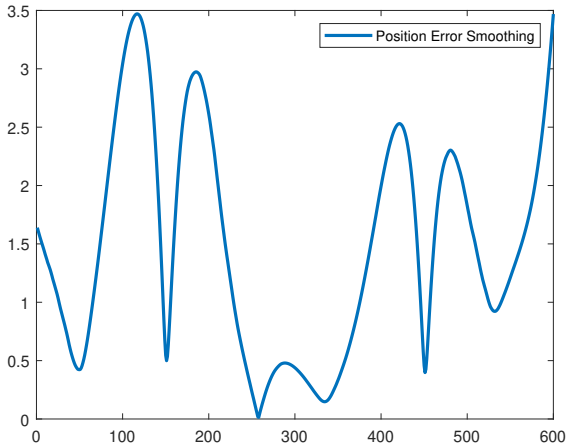


Figure 5: EKF smoothed error covariance

b)

Now for this question we add an outlier into our measurements at k=300 which is in the middle of curve. We observe that the filter's performance is quite off this time at k=300 since it deviates from the true states because of one single outlier in our measurements. However, looking at the smoothed results we can see that it did not get affect by the outlier to the same extent and it is estimates the true states quite well. This is a sell point for smoothing since it is quite robust to measurement noises. This is because as mentioned in smoothing we are not looking at the current measurement but also measurements in the future which lowers the affect of sudden large noise on a specific measurement on the total estimation. This observation can be seen in the follow figures but also in the error covariance figures as the peak of smoothed estimation is lower than the peak of filtered estimation.

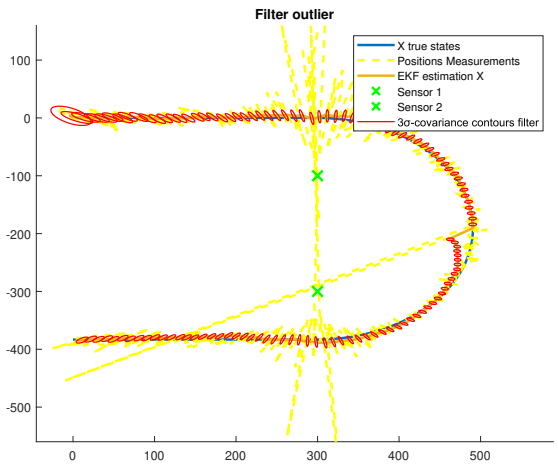


Figure 6: EKF - FILTERED with outlier at k=300

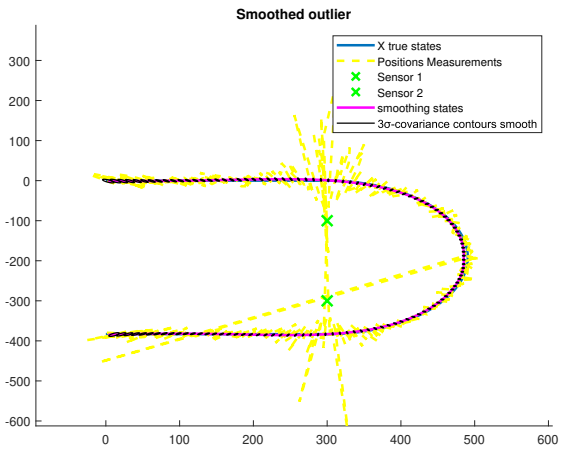


Figure 7: EKF - SMOOTHED POSITION with outlier at k=300

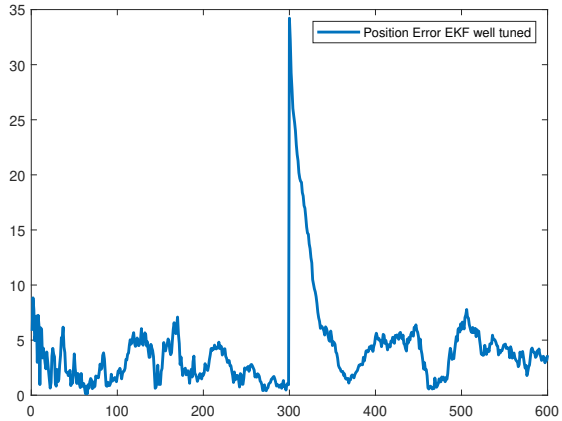


Figure 8: EKF- error covariance

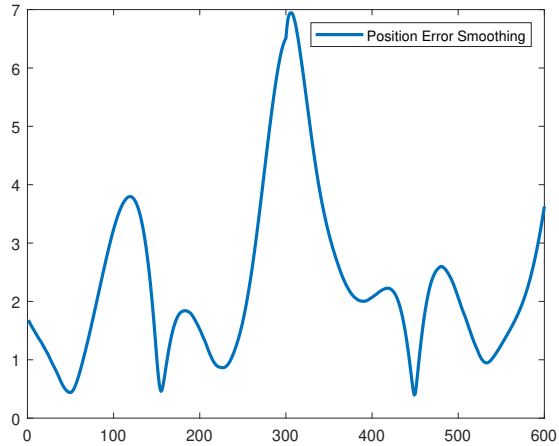


Figure 9: EKF- Smoothed error covariance

2 Particle filters for linear/Gaussian systems

a)

In this task, we compared the performance of the Kalman filter and the particle filter with and without resampling. Initially, the particle filter consistently under-

performed, but as we increased the number of particles considered, particularly to around 10,000, its performance became more competitive. Given that the system is linear Gaussian, the Kalman filter provides the optimal linear solution. Consequently, when we considered the resampled particle filter as our ideal solution, it consistently approached the estimates provided by the Kalman filter.

In summary, when the number of particles is low, the particle filter tends to perform worse compared to other variants, leading to significant errors in our estimates. While increasing the number of particles can improve performance, it escalates computational complexity to impractical levels. Resampling offers a superior solution, as illustrated in the figure below, by enhancing accuracy and mitigating the risk of particle degeneracy.

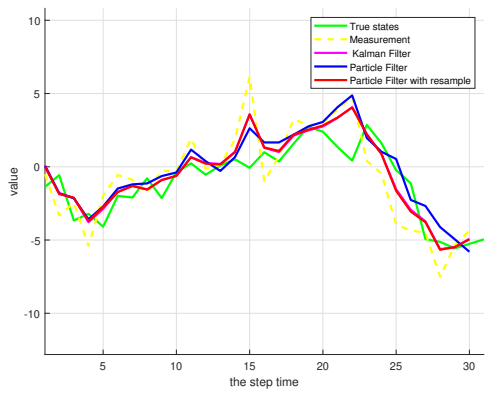


Figure 10: comparison between the different filters performance for N=1000

	N = 50	N = 100	N = 1000	N = 6000	N = 10000
Kalman Filter	0.9079	0.9079	0.9079	0.9079	0.9079
Particle filter without resampling	1.9256	2.4796	1.6020	0.7837	1.2795
Particle filter with resampling	1.0446	0.8472	0.9673	0.9299	0.9178

Table 1: MMSE values for different N

From the above table we can see the MSE for different amount particles. We can see that the particle filter without resampling only gets competitive results after we reach somewhere after thousands particles while the one with resampling starts to work well at N=100.

Now we can plot the error bars which is the error between our estimation and

true state we can see that the particle filter has quite the largest error variance out of the two. While the resampled particle filter and the kalman filter had a comparative performance.

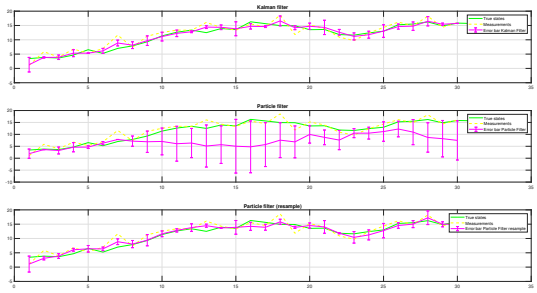


Figure 11: comparison between the different filters performance for N=10

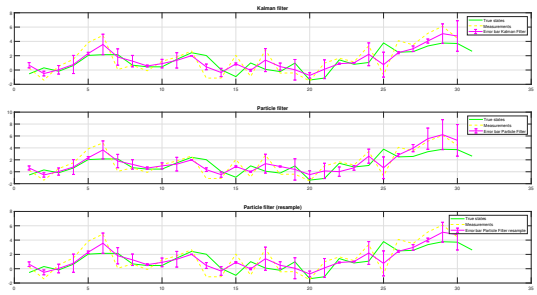
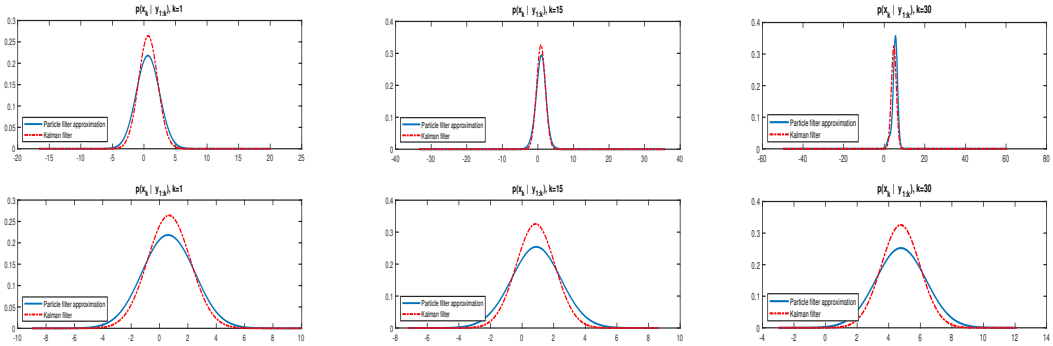


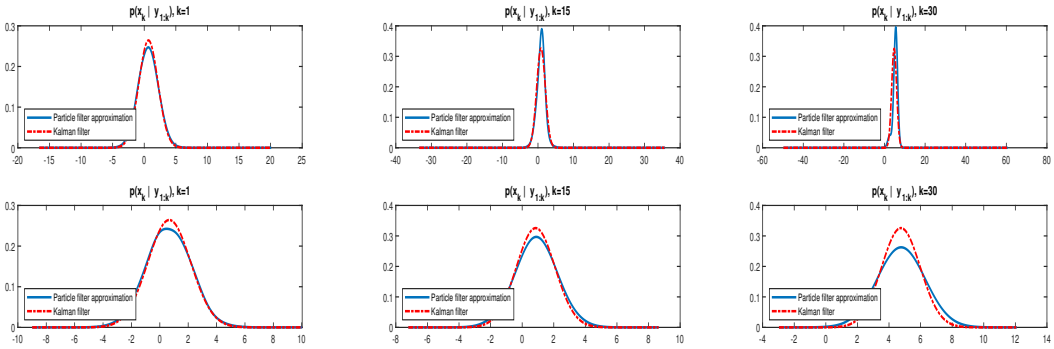
Figure 12: comparison between the different filters performance for N=10000

Now we can see the posterior desities of the two different variants of Particle filter for different time instances k=1, 15 , 30. The kernel width sigma that got tuned to the following values sigma1=0.5, sigma2=0.6, sigma3=0.9. Below we can see the plots for the initial values of sigma which is 1.



(a) k=1 *(b) k=15* *(c) k=30*

Figure 13: Posterior densities from two variants of Particle filters and the posterior density in Kalman filter before tuning the kernel size



(a) k=1 *(b) k=15* *(c) k=30*

Figure 14: Posterior densities from two variants of Particle filters and the posterior density in Kalman filter after tuning the kernel size

Several observations emerge from the analysis. Decreasing the kernel size results in narrower and more accurate visualization of the particles, providing clearer insights into their distribution across the space. Notably, in the case without resampling, reducing the sigma value correlates with a closer alignment between the particle filter posterior and the Kalman filter posterior. In conclusion, the performance of the particle filter is heavily influenced by the number of particles it considers. When this number is high, it becomes easier to fine-tune the filter to match the Kalman filter estimates. Conversely, when the number of particles is low, achieving this alignment becomes significantly more challenging.

b)

We know from previous assignements that kalman filter can handle wrong initial priors. However, for the particle filter it is observed that if there is no resampling the filter takes more time or might even never recover to the correct states because of the wrong prior. This is because wrong particles will stay affecting our estimation without any action to correct it. However, with the resampling the way the filter will work is that correct particles will stay relevant, canceling out the affect of the incorrect prior the more the time goes since we focus on the correct particles more with time. In conclusion resampling makes the particle filter handle wrong prior easier.

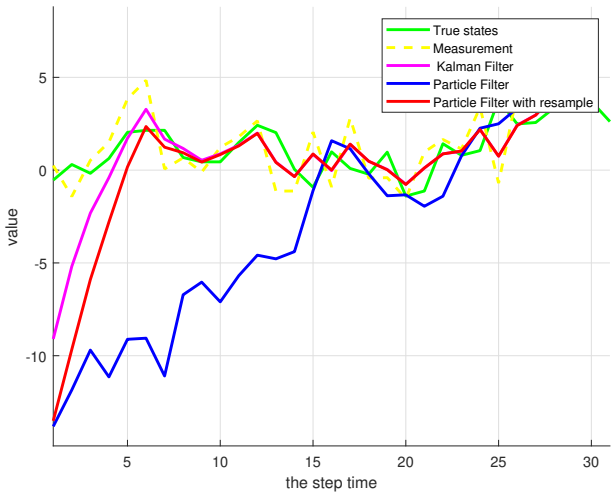


Figure 15: $p(x_0) = \mathcal{N}(x_0; -20, 2)$

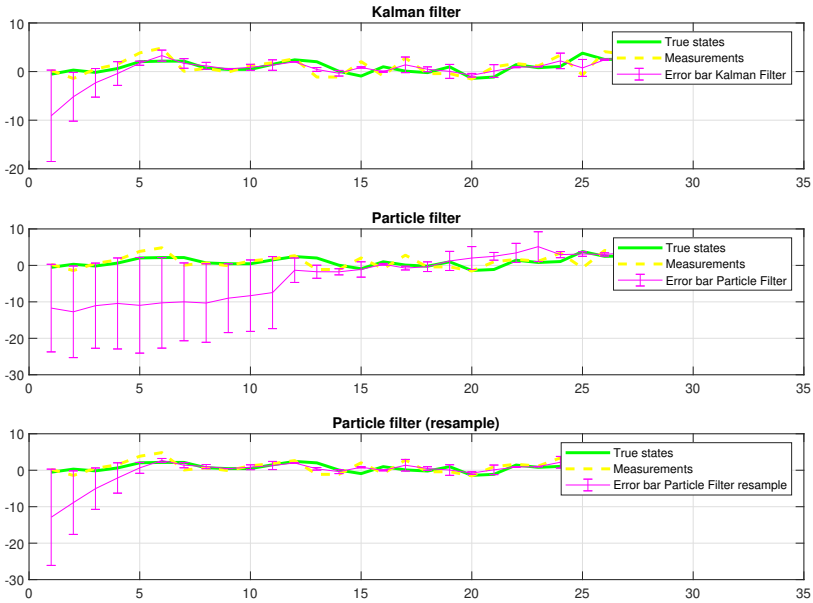


Figure 16: error bar for $p(x_0) = \mathcal{N}(x_0; -20, 2)$

c) and d)

Here we should examine how the particle filter works specifically how trajectories are generated. In the below figure we see for the case where w have no resampling. It is observed that the particles scatter in comparison to the true positions because of the variance of the process model. The problem with particle filter in general is degeneracy.Degeneracy refers to a situation where low weights get assigned to particles that are very far from the true states which with many iterations might mislead our particle filter to focus more on those areas which are totally off. This concentration of weights makes most of the particles ineffective in representing the posterior distribution accurately. Essentially, it reduces the effective sample size, leading to a loss of diversity among particles and a decrease in the quality of the filter's estimates since they deviate from the true states. Degeneracy typically occurs as the filtering process progresses, especially when the number of particles is limited or when the model is complex. It can hinder the performance of the particle filter by causing it to provide inaccurate or unreliable estimates of the state variables.

Resampling is a common technique used to address degeneracy in particle filtering. It involves replacing the particles with very low weights with particles that have higher weights, redistributing them to better represent the underlying distribution by focusing equally on the particles which have the highest weights. Afterwards all those particles will get equal weights assigned to them and only how high chance it is that the particle is the true estimate will affect how large effect it has on our prediction. This process helps maintain diversity among particles, preventing degeneracy and improving the accuracy of the filter's estimates. This method can be seen in how the PF with resample generated trajecotires close to the true states in comparinsion to the standard PF.

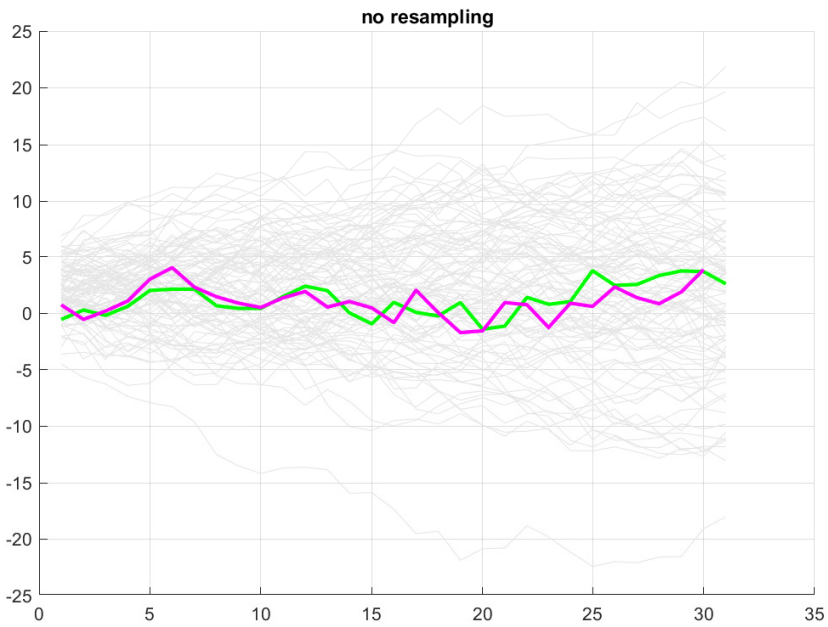


Figure 17: Particle filter no resampling, green is true states while the purple is estimated Kalman

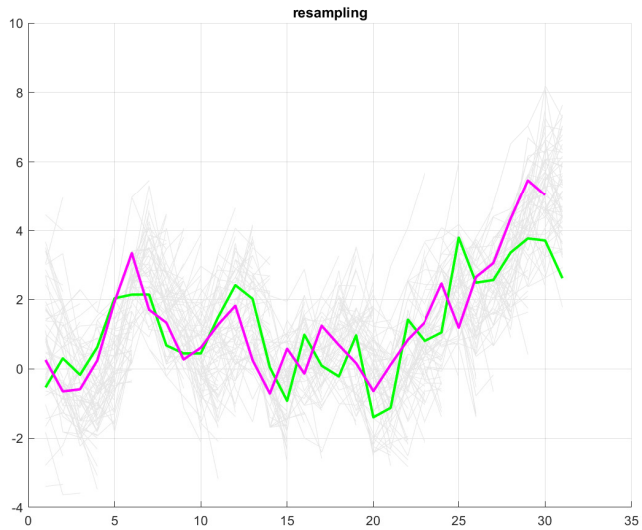


Figure 18: Particle filter with resampling, green is true states while the purple is estimated Kalman

3 Bicycle tracking in a village

The generated map and path can be seen in the following figure.

a)

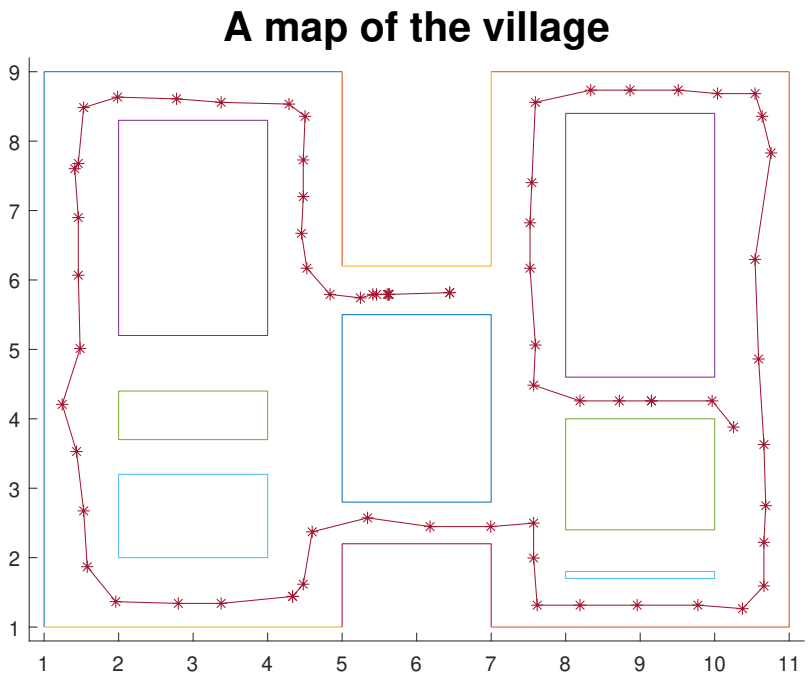


Figure 19: The generated map

b)

Velocity measurements can be seen in the following code.

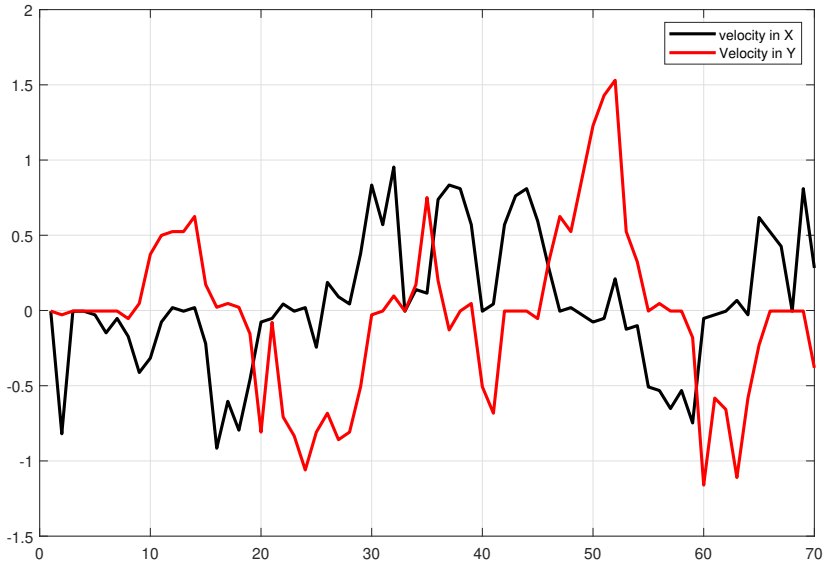


Figure 20: Velocity measurements

c)

In this question we should basically design a filter such that we should use the information from the map to estimate our positions that are not allowed to be inside buildings. Firstly, we need to know which motion model to use. The motion in the map fits two kind of models either CV or CT. Now CV is great for random walks and constant velocity and constant acceleration situations while CT is useful when we wish to orient an object in 2D or 3D while also modelling the translation of the object. The motion in such map of a bike will fit CT more than CV. However, CT is a nonlinear motion model hence it will be more computational intensive.
In the following equation we can see how the CV model looks

$$\begin{bmatrix} p_k \\ v_k \end{bmatrix} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_{k-1} \\ v_{k-1} \end{bmatrix} + \tilde{\boldsymbol{q}}_{k-1}$$

In the following equation we can see how the CT model looks and how it looks

in the discrete version.

$$\begin{bmatrix} \dot{x}(t) \\ \dot{y}(t) \\ \dot{v}(t) \\ \dot{\phi}(t) \\ \dot{\omega}(t) \end{bmatrix} = \begin{bmatrix} v(t)\cos(\phi(t)) \\ v(t)\sin(\phi(t)) \\ 0 \\ \omega(t) \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} q_c^v(t) \\ q_c^\omega(t) \end{bmatrix}$$

$$\begin{bmatrix} x_k \\ y_k \\ v_k \\ \phi_k \\ \omega_k \end{bmatrix} = \begin{bmatrix} x_{k-1} + Tv_{k-1}\cos(\phi_{k-1}) \\ y_{k-1} + Tv_{k-1}\sin(\phi_{k-1}) \\ v_{k-1} \\ \phi_{k-1} + T\omega_{k-1} \\ \omega_{k-1} \end{bmatrix} + \boldsymbol{q}_{k-1}$$

Now to know where to add the information of the map either in the measurement model or the motion model. If we were to integrate the map to our motion model it will demand more computations since it will become a constrained motion model, meaning the particles need to follow and fulfil constraints while propagating through the motion model. However, adding the map information into our measurement model is cheaper because it only requires updating the likelihood in our measurmeent model equations which is not complex. This can be seen in the following equations which are based on bayes rule where M is the map information whether "a particle is valid or not (inside a build or not)".

$$p(x_k|Y, M) = \frac{p(M|x_k, Y) \cdot p(x_k|Y)}{p(M|Y)}$$

$$p(x_k|Y, M) \propto p(M|x_k) \cdot p(x_k|Y)$$

To get the map information we can us the function isOnRoad.m as to know if a specific particle is on the road (valid) or inside a building (not valid). Only particles which are on the road will have a weight while other will have 0 as to not affect the estimation.

d)

Tried something in the code